

CSE416: TrueVision - Final Report

Team 3: Yoonseong Kim, Minsu Pak

Yoonseong.kim.1@stonybrook.edu || minsu.pak@stonybrook.edu

(<https://github.com/YoonseongKim2000/CSE-416-Team3.git>)

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Product description.....	4
1.2 Scope.....	6
What This Product Will Do.....	6
What This Will Not Do.....	6
Intended User.....	6
Target Platform.....	6
1.3 Users.....	7
1.4 User feedback.....	8
1.5 Existing alternatives.....	9
1.6 Definitions.....	10
2. Requirements.....	11
2.1 Functional Requirements.....	11
Section 1 - Nav Bar.....	11
Section 2 - Landing Page.....	11
Section 3 - Home/Analyze Page.....	11
Section 4 - Results Page.....	12
Section 5 - Pricing Page.....	12
Section 6 - Account Settings Page.....	13
Section 7 - Login Page.....	13
Section 8 - Sign Up Page.....	14
Section 9 - Purchase Page.....	14
Section 10 - Features Page.....	14
Section 11 - API Documentation Page.....	15
Section 12 - Minigame Page.....	15
Section 13 - Stretch Goals.....	15
2.2 Use cases.....	16
Use case 1: Submit Image.....	16
Use case 2: Evaluate Image.....	16
Use case 3: Select Model.....	16
Use case 4: Purchase a Paid Subscription.....	16
Use case 5: Re-generate API Key.....	17
2.3 Non-functional Requirements.....	18
3. System Architecture.....	19
3.1 Overview.....	19

3.1.1 Front End.....	19
3.1.2 Backend.....	19
3.1.3 Database.....	20
3.2 Data Design.....	21
3.3 UML Sequence Diagrams.....	22
3.4 API Design.....	24
3.5 Deployment.....	26
3.5.1 Front End Deployment.....	26
3.5.2 Back End Deployment.....	26
3.5.3 Database Deployment.....	26
3.5.4 Environment and Configuration Management.....	26
3.6 Code Conventions.....	28
3.6.1 General.....	28
3.6.2 Frontend.....	28
3.6.3 Backend.....	28
3.6.4 Database.....	29
4. Contributions.....	30
Yoonseong Kim.....	30
Minsu Pak.....	30
Together.....	30
5. Reflection.....	32
Minsu Pak.....	32
Yoonseong Kim.....	33

1. Introduction

1.1 Product description

As of 2025, AI tools have been used to create or modify images, and it has been reported that 34 million images are being generated daily (1). With an incredible volume of images created daily, along with the improvement in the quality of the generated image, one may begin to question the authenticity of the image they are viewing. The viewer could have various reasons why they would want the image they are looking at to be a real image made by a human.

1. The viewer may be a customer for a digital image, and they intend to use the digital image commercially. If the image has been made by AI, there is a chance that it could infringe on copyrighted material, and the viewer will be in legal trouble.
2. The viewer is reading a news article, and the article has been augmented with images. The images may have been generated to push a political agenda, and the viewer wants to be grounded.
3. The viewer is browsing artwork, and they want to support real artists and not people who use AI to pretend to be artists.

These are a few reasons why someone would like to know if an image is AI-generated or human-made. Overall, if someone needs to know if an image is AI-generated for any reason, we would like to provide them with a tool to assist in their investigation.

The tool we have decided to provide will be an image classifier trained on AI images and real images to be able to identify minute details that are not perceptible to the human eye. There will be two methods to access our tool; the first method will be through a website to access the classifier. The second method will be for users who are more adept with computers, because it will require them to acquire an API key from us to use our classifying service on their own platform.

3 major features will be implemented into this product. These features will be accessible through the website or our API.

1. The user can give an image to our product, and our product will identify if the image is AI-generated.
 - The image will go to our service, and then our machines will do the work to classify the image. The result of the analysis will be sent back to the user for them to review.
2. The user may select different models to get a better result.
 - There will be various models available to the user. Each model will be fine-tuned to certain styles. Currently, we are offering: general, art, and anime-style.

3. The user may “pay” for an API key to use our classifier on their own service.
 - Our tool will be provided for free through our website for the users. If the user likes our service, then they can have the ability to use our classifiers through an API.

1.2 Scope

What This Product Will Do

This product will provide users with a tool to analyze digital images and determine whether they have been generated by AI or created by humans. The tool will be built on classification models to detect features that distinguish AI-generated images from human ones. The users can access this functionality through a website or by integrating our tool into their own platform using an API we will provide. The product will also support multiple models (general, art, and anime-style) so that users can obtain more accurate results for their specific use case.

What This Will Not Do

The product will not edit, generate, or enhance images. It is only an image classifier. This product also does not guarantee a 100% accurate result, nor will it provide legal advice about copyrighted materials or the image's origin. The product will not moderate or restrict what type of images the user may upload. Additionally, this product will not support pictures of digital images.

Intended User

The primary users of this product include, but are not limited to:

- **Content consumers** who want to check the authenticity of the digital image found online. (e.g., social media posts, news articles, art platforms)..
- **Artists and art communities** who want to check the authenticity of a digital image when purchasing or judging artwork.
- **Businesses** that wish to integrate AI image detection on their platforms via the API (e.g., Art-hosting websites)

Target Platform

The product will be deployed as a web-based application for desktop users. The web-based application can be used on mobile platforms, but the UI will not be designed for mobile users. Additionally, an API service will be available for developers who need AI image classification in their own service.

1.3 Users

There are two primary groups of users for this product:

- **General Users:** This group of users can access the classifier through the website. This group of users may have limited experience with computers, so the website should be simple and intuitive. Their main goal should be to upload an image and then quickly find out if the image was made by an AI or a human. This group can include students, digital art enthusiasts, or general daily web users.
- **Advanced Users:** This group of users is more technically experienced. Their main goal will be to integrate our API into their service, so they will need documentation and an API endpoint. This group will consist of developers from businesses.

1.4 User feedback

Feedback from potential users and stakeholders will be most useful at two stages of development.

1. **UI Prototype Stage:** Once the web interface is implemented, but before the classification model is complete, it would be a useful point to receive feedback. The feedback we will collect at this stage will be on usability, clarity of results, and overall user experience. This information will let us know that the interface we have implemented needs improvement, or that there are elements the user liked. This information can be collected through a closed beta test with testing sessions with classmates and friends.
2. **Post-Development Stage:** After the service goes live, feedback will be focused on the accuracy of the model. The user will have a method to rate the result of the model on the website, and we can track the model's metrics through our service. With the feedback, we can focus on improving models that are not performing as well as they could be.

1.5 Existing alternatives

We have found two existing alternatives:

1. **Sight Engine** (<https://sightengine.com/detect-ai-generated-images>)
2. **Isgen.ai** (<https://isgen.ai/ai-image-detector>)

We found a quality in both of these websites that is both a strength and a weakness. Both websites offer many other services that check for other AI-related content. This makes their service have a lot more features, which can be considered a strength. However, if the focus on the website is to identify images, then the various services make it difficult to navigate back to the image identifier. Therefore, in our product, we will focus only on AI image identification and make it clear on the UI how to navigate back to the identification page.

Additionally, we have taken a look at their pricing page, and we will build our web product to be similar. Since this is a college project and not a real service, we will just pretend the payment has happened. We have also seen that the website does not show information if an image has been scanned on their platform before. Therefore, we will be implementing a method where we can track images and tell users how many times the image has been scanned before.

Furthermore, these existing services do not explicitly offer any AI API's for use. Therefore, our application will be offering this unique feature for users.

1.6 Definitions

- **AI-generated Image:** An image made or modified by artificial intelligence.
- **Classifier:** A machine learning model that analyzes data and then categorizes it into defined classes.
- **API (Application Programming Interface):** A set of functions that allow external software to interact with our system through programming.
- **False Positive/Negative:** A type of classification error.

2. Requirements

These are the requirements of TrueVision.

2.1 Functional Requirements

Section 1 - Nav Bar

- **Req 1-1:** The nav bar shall provide links to navigate
 - **Req 1-1.1:** The nav bar shall provide links to navigate to the home page, features page, pricing page, purchase page, mini-game page, and account settings page if the user is logged in
 - **Req 1-1.2:** The nav bar shall provide links to navigate to the home page, features page, pricing page, login page, mini-game page, landing page, and sign-up page if the user is not logged in

Section 2 - Landing Page

- **Req 2-1:** The landing page shall provide a navbar
- **Req 2-2:** The landing page shall provide a button to direct to the analyze page
- **Req 2-3:** The landing page shall provide information about the website and its services
- **Req 2-4:** The landing page shall provide a button to redirect to the purchase page

Section 3 - Home/Analyze Page

- **Req 3-1:** The home page shall display the image that has been uploaded
 - **Req 3-1.1:** The home page shall display the name of the image file
- **Req 3-2:** The home page shall allow users to upload an image from their device
- **Req 3-3:** The home page shall allow users to click a button to begin image analysis
- **Req 3-4:** The home page shall provide a navbar
- **Req 3-5:** The home page shall allow users to select the AI model they would like to use for the image analysis if they are a paid user

Section 4 - Results Page

- **Req 4-1:** The results page shall display the image that has been analysed
- **Req 4-2:** The page shall link back to the home/upload page
- **Req 4-3:** The page shall display whether the analysis believes the image is real or AI-generated
- **Req 4-4:** The page shall display the confidence percentage of the analysed result
- **Req 4-5:** The results page shall display a navbar
- **Req 4-6:** The results page shall allow users to view an AI attention heatmap overlay over the analysed image if they are a paid user
 - **Req 4-6.1:** The results page shall allow users to turn off and on the heatmap overlay
- **Req 4-7:** The results page shall display the user's scan history
 - **Req 4-7.1:** The results page shall not display the user history section if the user is not logged in
 - **Req 4-7.2:** The results page shall display "no history" if the user has no history
 - **Req 4-7.3:** The history card shall display the image, the name of the image file, the classification result, the confidence result, and the model used
 - **Req 4-7.4:** The user's scan history shall persist through sessions

Section 5 - Pricing Page

- **Req 5-1:** The pricing page shall provide a navbar
- **Req 5-2:** The pricing page shall provide buttons to purchase a paid subscription
 - **Req 5-2.1:** If the user is logged in, the button shall redirect to the payment page
 - **Req 5-2.2:** If the user is not logged in, the button shall redirect to the login page

- **Req 5-3:** The pricing page shall provide information about the paid subscription and its services

Section 6 - Account Settings Page

- **Req 6-1:** The account settings page shall display the user's email linked to their account
- **Req 6-2:** The account settings page shall allow users to delete their accounts
- **Req 6-3:** The account settings page shall save any changes to the user's account
- **Req 6-4:** The account settings page shall allow users to change their password
 - **Req 6-4.1:** The page shall require users to input their current password to change their password
 - **Req 6-4.2:** The page shall allow users to input their new password
 - **Req 6-4.3:** The page shall allow users to confirm their password change to change their account password
- **Req 6-5:** The page shall display the user's current pricing plan
 - **Req 6-5.1:** The page shall allow users to change their current pricing plan by directing them to the payment page
- **Req 6-6:** The page shall allow users to re-generate their API key
- **Req 6-7:** The page shall allow the user to cancel their plan
- **Req 6-8:** The page shall display the current tokens they own
- **Req 6-9:** The page shall have a button to hide/reveal the user's API key
- **Req 6-10:** The page shall have a copy to clipboard button for the user's API key

Section 7 - Login Page

- **Req 7-1:** The login page shall provide a nav bar
- **Req 7-2:** The login page shall provide an area to input the user's account email
- **Req 7-3:** The login page shall provide an area to input the user's password
- **Req 7-4:** The login page shall have a button to submit information to log in
- **Req 7-5:** The login page shall provide a link to the 'sign up page'
- **Req 7-6:** The login page shall check if the email and password combo is valid

- **Req 7-7:** The login page shall redirect to the home page if the login was successful

Section 8 - Sign Up Page

- **Req 8-1:** The sign-up page shall provide a nav bar
- **Req 8-2:** The sign-up page shall provide the user with a method to enter their email address
- **Req 8-3:** The sign-up page shall provide the user with a method to enter their desired password
- **Req 8-4:** The sign-up page shall provide the user with a method to enter their password again to confirm they typed it correctly
- **Req 8-5:** The sign-up page shall provide the user with a method to read the “terms and conditions”
- **Req 8-6:** The sign-up page shall provide a link to the login page
- **Req 8-7:** The sign-up page shall check if the email is unique
 - **Req 8-7.1:** If unique, the login page shall create the user account.
 - **Req 8-7.2:** If not unique, the login page shall notify the user of the issue.
- **Req 8-8:** The sign-up page shall provide a button to submit the sign-up details entered
- **Req 8-9:** The sign-up page shall redirect to the login page upon signing up
- **Req 8-10.0:** The sign-up page shall provide confirmation that the account has been made if all required fields, the email and password, are valid.
- **Req 8-10.1:** The sign-up page shall provide an alert if any of the required fields are not valid

Section 9 - Purchase Page

- **Req 9-1:** The purchase page shall provide a nav bar
- **Req 9-2:** The purchase page shall provide buttons that simulates purchasing
- **Req 9-3:** The purchase page shall allow users to purchase 50, 100, or 200 tokens

Section 10 - Features Page

- **Req 10-1:** The features page shall provide a nav bar
- **Req 10-2:** The features page shall provide information regarding the available models

Section 11 - API Documentation Page

- **Req 11-1:** The documentation page shall provide a nav bar
- **Req 11-2:** The documentation page shall provide a tab to navigate to sections of the documentation

Section 12 - Minigame Page

- **Req 12-1:** The page shall offer 3 models to play against
- **Req 12-2:** The page shall display the challenge image
- **Req 12-3:** The page shall provide buttons to select if an image is AI or Human
- **Req 12-4:** The page shall notify when the AI is guessing
- **Req 12-5:** The page shall keep track of the number of rounds
- **Req 12-6:** The page shall keep track of the number of user wins
- **Req 12-7:** The page shall keep track of the number of AI wins
- **Req 12-8:** The page shall display the AI's attention
- **Req 12-9:** The page shall provide a select another model button
- **Req 12-10:** The page shall provide a play again button

Section 13 - Stretch Goals

- Labeling and tracking uploaded images to give statistics to the user
- Allow image links to be used instead of uploading an image
- Login in with Google authentication

2.2 Use cases

Use case 1: Submit Image

Actor: User

Priority: Essential

Primary Scenario:

1. The user clicks on “select file”
2. The user selects a file
3. The user clicks on upload
4. The file is sent to the backend

Extension Scenario:

2a. The user selects a file that is not an image, and the website rejects it.

Use case 2: Evaluate Image

Actor: Backend

Secondary Actor: AI model

Priority: Essential

Primary Scenario:

1. The backend receives a file
2. The backend checks if the file is an image
3. The backend preprocesses the image
4. The backend passes the image through an AI model
5. The backend sends the result back to the user

Extension Scenario:

2a. The backend sends the user an error message

Use case 3: Select Model

Actor: User

Priority: Essential

Primary Scenario:

1. The user gets an option to pick a model
2. The user selects a model
3. The choice is saved on the website

Extension Scenario:

2a. If the user is not logged in, the user cannot select a model

Use case 4: Purchase a Paid Subscription

Actor: User

Secondary Actors: Backend, Database

Priority: Expected

Primary Scenario:

1. The user is logged in
2. The user clicks to purchase the paid plan
3. The user completes the steps to purchase the paid plan
4. The backend receives data that a user has purchased a paid plan
5. The backend authenticates that the user's account is real and that their account information is valid through the data stored in the database
6. The database information is altered to show that the user is a paid user
7. The user is shown that their purchase was successful

Extension Scenario:

- 1a. If the user is not logged in, redirect to the login page
- 5a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. The payment and change of account status are cancelled.

Use case 5: Re-generate API Key

Actor: User

Secondary Actor: Backend, Database

Priority: Nice to have

Primary Scenario:

1. The user is logged in and has a paid subscription
2. The user navigates to their account settings page
3. The user is presented with information regarding the management of the API key
4. The user clicks a button to re-generate their API key
5. The backend receives data that the user wants to re-generate their API key
6. The backend authenticates the user's account is real and that their account information is valid through the data stored in the database
7. The backend generates a new API key
8. The backend changes the user's API key in the database
9. The new key is sent to the website
10. When the key information is received, the website is updated to show the new API key

Extension Scenario:

- 6a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. Change of account status fails and it is cancelled.

2.3 Non-functional Requirements

Category	NFR	Priority	Achieved
Performance	Each image shall not take longer than 15 seconds to evaluate	High	O
	Each model shall have an F1 score > 0.85	High	O
Cost	There shall be no financial need for any part of the project	High	O
Security	The website shall use HTTPS	High	O
	Raw passwords shall not be stored in the database; they must be hashed	High	O
	Logged-in users will be logged out after a period of time and must log in again.	Medium	O
	User authentication must occur whenever sensitive actions take place (change of password, purchase of subscription, etc.)	High	O
Maintainability	Maintenance downtime should be limited to under 1 hour	Low	?
	Major implementations shall be modularized and follow a predetermined interface	High	O
Reliability	The service shall be available to the user unless notified of updates and/or maintenance.	Medium	?

3. System Architecture

In this section, all relevant information on how the project will be made and deployed can be found.

3.1 Overview

For the TrueVision project, many components are working together. Simplified, here is a list of components required to bring TrueVision to life:

- Front End
- Back End
- Database

3.1.1 Front End

The front end of TrueVision serves as the main interface that users will interact with the system. It is responsible for providing a smooth and intuitive experience for both general users and advanced users accessing the API documentation. The front end will be made using **React**, a JavaScript library that enables a streamlined development process by combining HTML and JavaScript into one file. Additionally, Vite will be used to accelerate build time, so development of the front end can be optimized. Lastly, the code will be using **TypeScript** to ensure there is type safety and that runtime errors are reduced.

For styling, **CSS** and **Bootstrap 5** will be used to ensure our design remains consistent across pages. Bootstrap's built-in grid system and premade components will be utilized to make the layout adaptable to different screen sizes. However, the target platform is desktop browsers; therefore, screen sizes that belong to mobile devices will not be as polished. Navigation between pages, such as the homepage, pricing page, and so on, will be handled using **React Router Dom** for transitions without full page reloads.

The front end will communicate with the back end using **HTTPS** requests using the built-in **fetch** feature in TypeScript. This will allow users to upload images to receive classification results, manage their accounts, or use our API by interacting with the back end's **RESTful API**.

3.1.2 Backend

The back end of TrueVision is responsible for managing user requests, running AI image classification models, handling database operations, and maintaining secure communication. It is implemented using **Python 3.11.0** and **FastAPI** for asynchronous

operations and automatic documentation. The automatic documentation is generated via **Swagger UI**, and it makes it convenient for testing.

The back end is structured into separate modules for maintainability. The routes package we created contains API route definitions for core functionalities. These functions can include, but are not limited to:

- User Authentication
- Database operations
- Classification prediction

The classifier component of our back end uses PyTorch and the **Hugging Face Transformers** library. The transformers library is used to load a **Vision Transformer** and fine-tuned weights. Uploaded images are preprocessed using **OpenCV**, **Pillow (PIL)**, and **Torchvision** transforms to standardize input before being passed to the model. The model outputs probabilities indicating whether the uploaded image was AI-generated or human-made, which are then returned to the user as a **JSON** response.

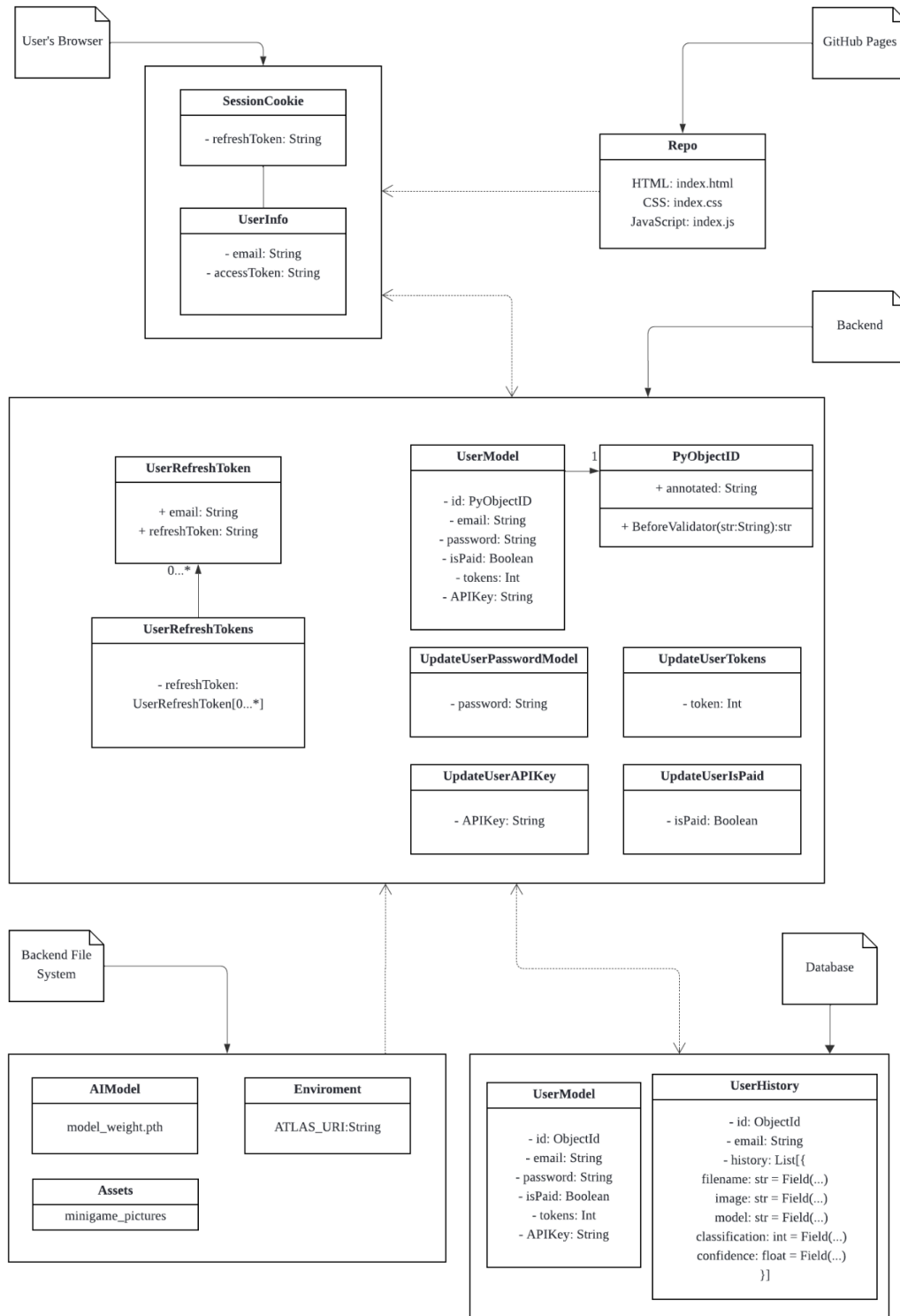
The backend communicates with the MongoDB Atlas database using MongoDB's **PyMongo** driver and its **AsyncMongoClient** function to interface with FastAPI's asynchronous functionality. Additionally, the **PyDantic** library is used to parse between the BSON format used by the database and the JSON format used by FastAPI, and for type validation between MongoDB and Python.

The back end also utilizes **FastAPI's CORSMiddleware** to enable communication between the front end and the **FastAPI** back end. Sensitive back end configuration data, such as database URIs and API keys, is stored separately in environment variables. Additionally, the back end shall utilize **Uvicorn 0.37.0** along with **OpenSSL** to enable **HTTPS**.

3.1.3 Database

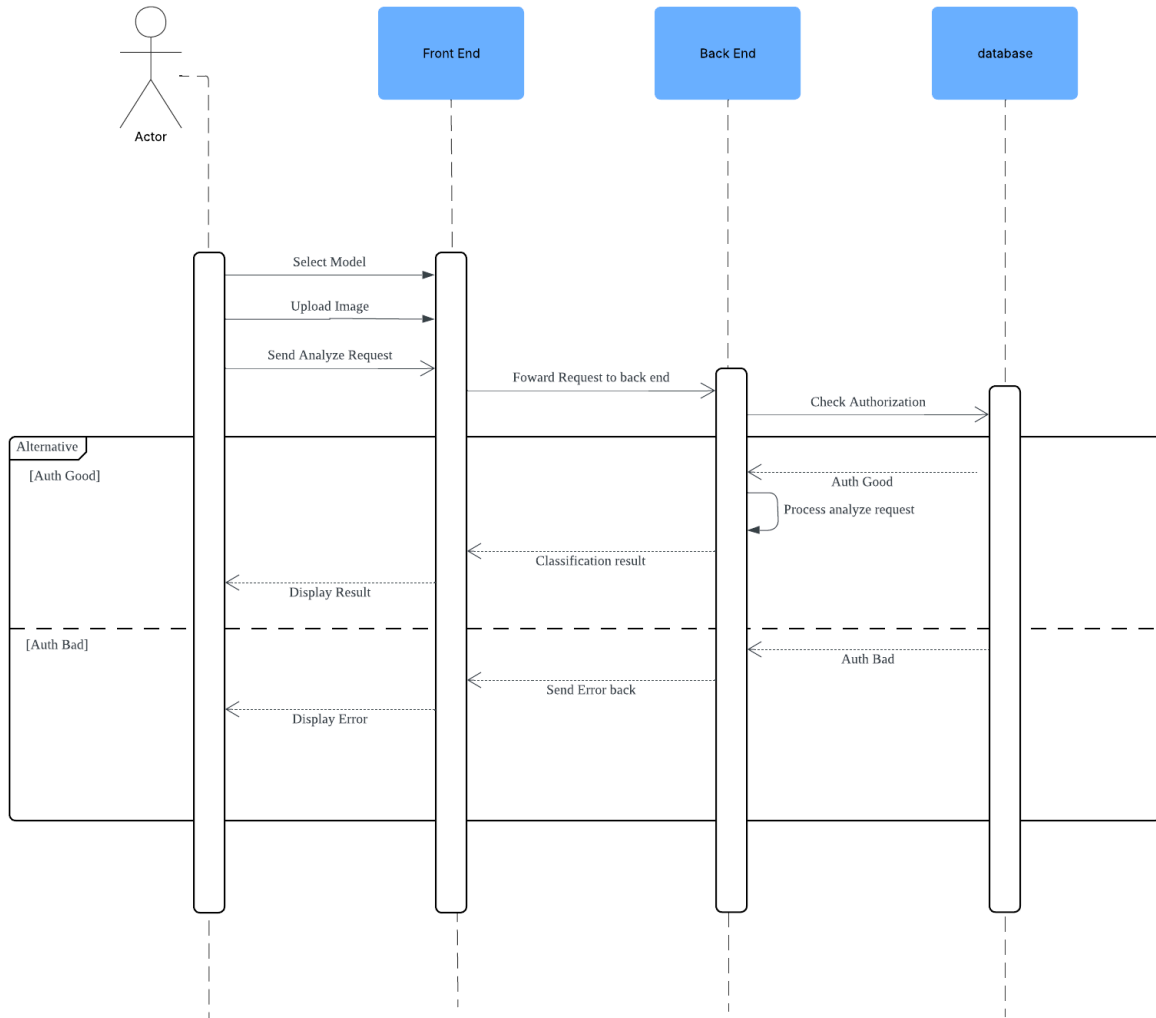
The database for TrueVision uses **MongoDB** hosted on the **MongoDB Atlas** cloud service. It is responsible for securely storing the persistent data required by our service, and handles **CRUD** operations to and from the backend server. MongoDB was initially chosen for its non-relational features and flexibility of schemas that were best suited for our first project idea. This database model was retained for our new project as such features were still useful to implement. Currently, the database is used to store user information for user operations on the front end.

3.2 Data Design

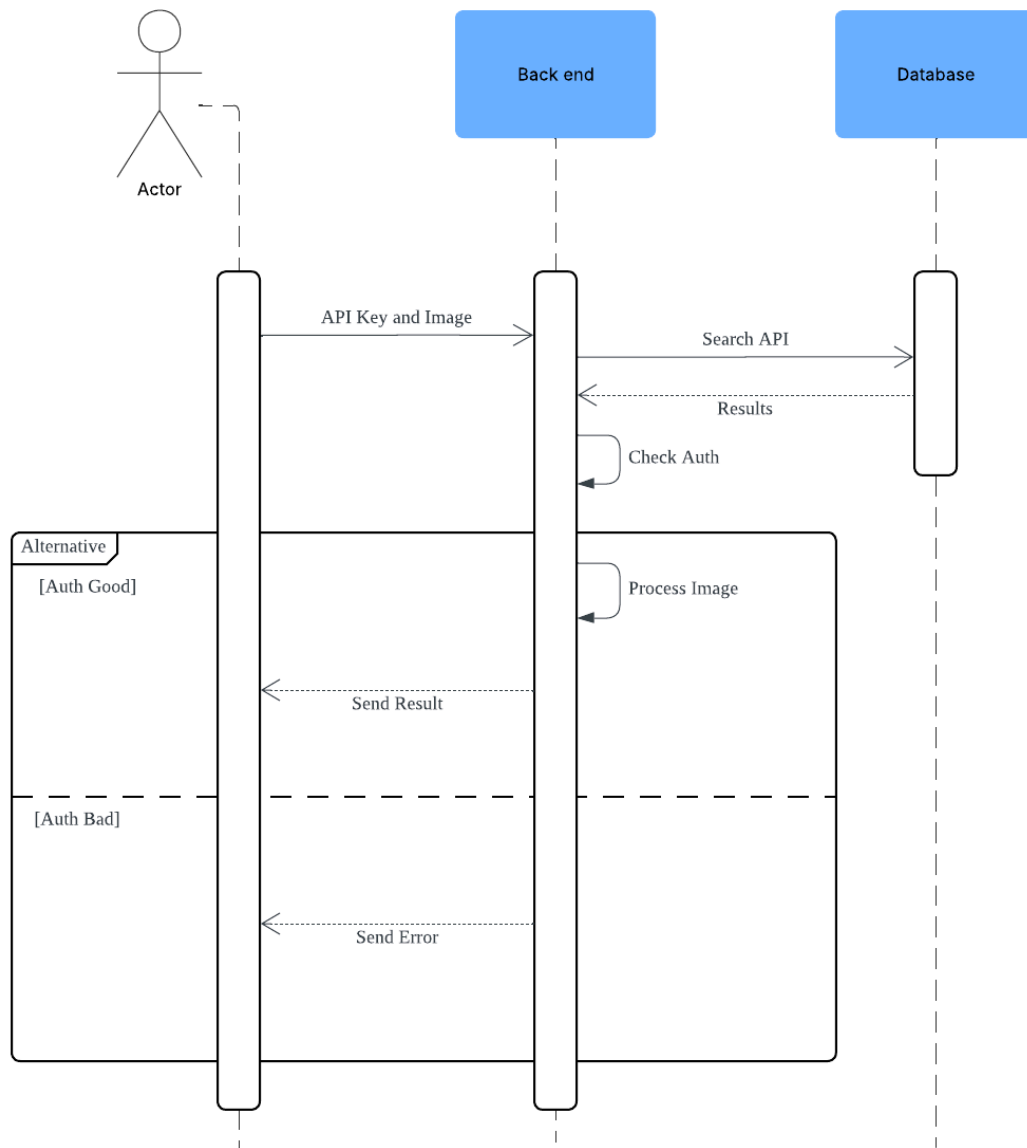


3.3 UML Sequence Diagrams

User Uses Analyse Image Feature



Advanced User API Call



3.4 API Design

Main URL: <https://220.126.157.140:10000>

Name	HTTP Method	URL	Input	Output
Login	POST	/access/login	String: email String: hashedPassword	String: accessToken Cookie: refreshToken Status: issues
Sign Up	POST	/sign_up	String: email String: hashedPassword	String: email String: isPaid Int: tokens String: APIKey Status: issues
Analyze Image Web	POST	/api/analyze	String: model File: image	Int: classification Float: confidence B64 String: originalImage B64 String: heatMap B64 String: masked
Analyze Image API	POST	/api/APianalyze	String: model File: image String: APIKey	Int: classification Float: confidence B64 String: originalImage B64 String: heatMap B64 String: masked
Change Password	PUT	/user/password	String: accessToken String: email String: new PasswordHash	Int: numberModified Status: issues
Purchase Plan	POST	/purchase/monthly	String: accessToken	Boolean: result Status: issues
Cancel Plan	POST	/user/cancelPlan	String: accessToken	Boolean: result Status: issues
Delete Account	DELETE	/user/delete	String: email String: hashedPassword String: accessToken	Boolean: result Status: issues
Start Minigame	GET	/api/minigame/start	String: model	UUID: roundID String: imageURL

Get a round image	GET	/api/minigame/image	String: roundID	B64 String: image
Submit a guess and scan image	POST	/api/minigame/guess	String: roundID Int: guess	Int: truth Boolean: humanCorrect Boolean: aiCorrect, Int: aiPrediction Float: confidence B64 Str: original_image B64 Str: attention_heatmap B64 Str: masked_overlay
Tier Info	GET	/user/tierInfo	String: accessToken	Boolean: isPaid
Get User History	GET	/user/get_history	String: accessToken	HistoryRecord[] : [{ filename: string; image: string; model: string; classification: number; confidence: number; }]
Update User History	POST	/user/update_history	String: accessToken Dict: { filename: string; image: string; model: string; classification: number; confidence: number; }	JSON: {"message": "History updated"} Or Status: issues
Add 50 Tokens	POST	/puchase/50	String: accessToken	Status: issues
Add 100 Tokens	POST	/puchase/100	String: accessToken	Status: issues
Add 200 Tokens	POST	/puchase/200	String: accessToken	Status: issues
Regenerate Key	PUT	/user/regenKey	String: accessToken	Status: issues
User Info	GET	/user/info	String: accessToken	Boolean: tier Int: tokenRemaining String: apiKey

3.5 Deployment

Deployment for TrueVision is designed to be kept simple and functional for demonstration purposes. Since this is a university project, and our team does not have the funds to utilize industry tools, we have looked for free methods of deployment. The system will be deployed as a web application consisting of a front end, a back end, and a database.

3.5.1 Front End Deployment

The front end will be hosted using **GitHub Pages**. It is free and provides a way to reliably host a static site. Using this method is convenient because we are already using **GitHub** to version control our project. Therefore, we only needed to create a branch of our main project and push the compiled **React** project. TrueVision will be accessible via the **GitHub Pages** subdomain assigned to the repository (<https://yoonseongkim2000.github.io/CSE-416-Team3/>).

3.5.2 Back End Deployment

The back-end server will be deployed on a self-hosted environment, running directly on a desktop machine we own. The server uses **Uvicorn 0.37.0** as the ASGI server for running the FastAPI application. The backend operates over **HTTPS** using a self-signed SSL certificate generated with **OpenSSL** on a **WSL (Ubuntu)** environment. This setup ensures **HTTPS** communication between the frontend and backend during data transfers, particularly when users perform authentication actions. If this were a real production project, then a domain could be purchased, and then a legitimate certificate from a CA could be acquired.

The machine hosting the backend also contains the **Vision Transformer (ViT)** model weights, which are loaded locally at runtime by the FastAPI service. This allows the image classification functionality to run entirely on the host machine without requiring external compute services or additional network dependencies.

3.5.3 Database Deployment

The database will be deployed using the **MongoDB Atlas** cloud service. The service is being used with a free account, and provides enough storage and performance to run our website's database operations. The actual server is being run by 3 **AWS** instances in Seoul.

3.5.4 Environment and Configuration Management

TrueVision has some sensitive credentials, such as database connection strings and SSL certificates. These are stored offline and locally with the backend. The environment

variables are defined in a local **“.env”** file. Additionally, the SSL certificate and key, along with the model weights, are stored locally. Relevant information is loaded onto the server upon boot and is not accessible through our back end.

3.6 Code Conventions

This project does not follow a set coding convention; rather, we have developed a system that makes collaboration possible. Here, you can see our short but practical set of conventions for readability and consistency.

3.6.1 General

- Use meaningful and descriptive names. We prefer understanding what something is over brevity.
- Keep functions small and modularized where possible.
- Secrets must never be in the code or committed to GitHub. Use environments.
- Add comments to functions that will be used by others.
- Develop locally and push to pages and the repository when significant progress is made.
- Leave notes in the repository about key functions.

3.6.2 Frontend

- Files and Components
 - Components will use PascalCase
 - Hooks will use camelCase
- Identifiers
 - State variables and handlers will use camelCase
 - Constants will use camelCase
- Styling
 - CSS class names are kebab-case, custom CSS or overrides will use PascalCase
 - CSS IDs are snake_case
 - Make a CSS file for each page
- Formatting
 - Use tabs to signify the depth of TypeScript or HTML
- Type Safety
 - Use TypeScript types and interfaces for component props and functions
 - Avoid using “any” where possible
- Accessibility
 - Attempt to provide alt text

3.6.3 Backend

- Files and Modules
 - Files and modules are snake_case
 - Constants are SCREAMING_SNAKE_CASE

- API route URLs are in snake_case
- Identifiers
 - Functions and variables use snake_case
 - Classes follow PascalCase
- Types
 - When possible, include function input types
- AI/Model Code
 - Keep models global and do not change the device or use model.train() anywhere
 - Use a GPU when possible

3.6.4 Database

- The database follows the generally accepted MongoDB naming conventions. This includes:
 - Collections in snake_case
 - Fields in camelCase
 - Indexes in snake_case

4. Contributions

This section lists what each member did. To give a general overview, both members worked on elements of the frontend and backend code, whilst Minsu focused on developing the database and Yoonseong on developing the AI models for the project.

Yoonseong Kim

- Experimented with my home network and hardware to deploy the backend
- Procured 2 datasets for the AI model and semi-created the third dataset
- Set up the GitHub Pages for the frontend deployment
- Created the game page
- Created and set up the basic design on numerous frontend pages for Minsu to refine and complete
- Fixed a critical issue where the frontend would not communicate with the backend because the backend was communicating in HTTP; accomplished by creating a self-signed certificate and key to enable HTTPS mode for the backend
- Refactored an old CSE 316 project for API presentation demo
- Figured out how to create routes for our backend so the APIs can be separated
- Created realistic content for our documentation page
- Set up scripts and development environment

Minsu Pak

- Set up the MongoDB Atlas cloud database
- Developed the backend code to interface the Python backend with the MongoDB database, and allow communication between the two
- Designed the GUI design concepts for the website
- Set up user-related functionality (e.g., creating accounts, logging in, etc.) on the front and backend
- Developed the authentication functionality for the frontend and backend checking features
- Created the required database functions for the backend
- Refined the website's GUI
- Made the logo and mascot images
- Updated Jira and kept the team on track

Together

Within this project, there were a lot of tasks that needed to be completed. Some tasks were very difficult and took a long time to complete, whilst some tasks were easy and quick to finish. Therefore, within this project, Yoonseong took care of the numerous easy

tasks while Minsu took on the harder ones. Overall, the time spent on the project is equally split between the two group members.

Below are the tasks that was completed coopertaively:

- Connecting APIs and API calls
- Documentation, presentations, and diagrams
- Solving cookies & CORS issues
- Frontend webviews
- Testing and resolving bugs
- Paid feature functionalities

5. Reflection

Minsu Pak

When I began this group project, I expressed that I wanted to get some experience working on the database, as, in our previous development project, Yoonseong had done most of the database operation. Other than this I did not have any strong opinions on what our project should be. I think this mindset eventually helped us to realistically assess our ideas and adapt to changing requirements. Although I was quite fond of our initial idea for the TRPG service, we eventually agreed that it seemed like too much work given only a semester as well as our workload for the semester. Thus, we had to quite quickly pivot, and come up with a project plan and specifications/requirements for our new idea. Situations where we had to suddenly switch a major aspect of our project also occurred when we realised that the freemium service we were using for our backend was not adequate, and instead began to set up Yoonseong's PC as a backend. Although not ideal, I do think that such experiences can occur in real-life development, and this also allowed me to get a little bit of experience with different services, such as Flask. Overall, I am pretty satisfied with our completed project. I do wish I had been less busy this semester, so I could have implemented more of our stretch goals such as Google authentication, or tracking uploaded images to save processing the same image and to track statistics. Making time to work on this project was also my biggest challenge throughout the course. Seeing the other groups' websites, I did feel like our project was a little too simple. However, even during the final week, we couldn't really think of any additional functionalities that we could get done in time. Nonetheless, we completed all the features and goals we set out to include during planning, so I would like to think we were being cautiously realistic about our project. Additionally, since this was the second time I had done a software development project with Yoonseong, we had no major issues, and each of our roles in development were pretty clear, making the collaborative experience quite streamlined.

As for the course itself, although I had learnt the major stages of the software development cycle, and also the different practices and documentation types used in the stages during CSE316, this course felt slightly more in-dept with the documentations and strategies we learnt. For example, I did not previously know about UML sequence diagrams or how to document use cases. In fact, during development, I actually found a UML sequence diagram in the FastAPI documentation. Another aspect of this course I found helpful was the frequent presentations. Before, I was not very confident in public speaking. However, after doing so many presentations during this course, I am now more comfortable with presentations.

Yoonseong Kim

In this project, there are many things that I have realized by the end. The first one is that I understand documentation can be very helpful. I will admit that it is somewhat an annoyance to draft and complete but once it is done, it is an excellent reference you can come back to during development. However, this will only hold true if me and Minsu makes an effort to create good documentation. Another thing I realized quite early on in the project was how much work our initial project would have been. I must admit, it would have been amazing if we could have pulled off our original idea but with the constraints we had, that would not have been possible. To change our project, I let Minsu know that I know how to train vision based AI models and then we can implement our joke idea we had over the previous semester. That project was making a AI model where it can detect AI vs non-AI images. With our team, I think this suggestion was well suited for our group because I think the project "TrueVision" played more into our strengths. I had the foundational knowledge to train models that we need and Minsu had the design abilities to conjure a website from this base idea. Therefore, if we had to do another project like we had in CSE 416, then the first thing to consider would be the size of the team, our expertise, and time available.

There were various challenges that we faced as a group. We as a team, did not want to spend any money for our project. Therefore, Minsu and I were tasked with finding ways to deploy everything before we began work. Minsu told me about GitHub Pages and we quickly confirmed that would work. Then I went to look for a viable option for a backend. In the beginning, I found Render and it seem like an ok choice. I went to work on deploying the backend as a test but I quickly found out that the hardware for our free backend service was too slow to support what we want to do. Due to this issue, I had to learn very basic network configuring to that I can open the correct port on my router's firewall. Once the backend was visible on the internet, I tried to make the frontend talk to it but there was an issue because the connection was not secure. To resolve this, I had to learn how use openssl to create a private key and self signed certificate. This has mostly resolved the problem but because the certificate was not signed by a certificate authority, all the browsers flagged it as untrusted. Despite the networking problems, I feel like I have learned a lot about how systems are deployed in real life and how to address these issues. Hopefully, when I am working for a company that needs a back end deployed, I can spend their money to buy a domain and get a certificate signed by a CA. There are many things that I have learned due to development issues but on my end, I think those were the most valuable lessons I learned.